Hit Song Popularity Prediction (Regression)

Hit Song Popularity Prediction using Audio Features

Predicting track popularity scores (0-100) using acoustic signals and machine learning regression models.

GITHUB REPOSITORY: https://github.com/BlueMist08/AI-Project-sample

Course: AI101 | Team No: 17 | Semester: Autumn 2026

Documentation Lead

Aanya Sharma

Entry: 2026AIB1001

README architecture, repo structure, clear documentation, commit history curation.

*** Contributed code to repository**

Execution & Reproducibility

Aaditya Chandol

Entry: 2026AIB1102

Environment configuration, requirements.txt, automated pipeline runner (main.py).

*** Contributed code to repository**

PPT & Visualization Lead

Simran Kaur

Entry: 2026AIB1203

5-slide presentation deck, diagnostic residual plots, metric comparison charts.

*** Contributed code to repository**

Video & Demo Lead

Kabir Singh

Entry: 2026AIB1304

Jupyter notebook walkthrough (.ipynb), pre-computed outputs, video recording lead.

*** Contributed code to repository**

Problem Statement and Dataset Overview

Problem Definition: What & Why?

The Challenge:

With over 100,000 new songs released daily, record labels and independent artists need quantitative insights to forecast commercial reception and track popularity.

The Objective:

Formulate song popularity prediction as a supervised regression task. Use intrinsic acoustic features from Spotify's audio analysis API to predict popularity scores (0 to 100).

The Research Question:

Can intrinsic acoustic properties (tempo, energy, acousticness) alone determine if a song becomes a hit, or does musical success require genre and cultural context?

Evaluation Focus:

Model performance evaluated via R2 score (variance explained), RMSE (error penalty for large deviations), and residual diagnostics to test regression assumptions.

Dataset Profile & Acoustic Attributes

Dataset Source:	Spotify Tracks Dataset (Kaggle: yashdev01 / maharshipandya)
Dataset Scale:	114,000 raw tracks across 114 diverse genres
Deduplicated Set:	89,740 unique tracks (duplicates removed across genre tags)
Target Variable:	Popularity (Continuous integer: 0 to 100, Mean=33.2, Std=20.6)
Split Strategy:	80% Training (71,792 tracks) / 20% Test (17,748 tracks)

Key Audio Features Selected:

* **Danceability:** Rhythm stability, tempo, beat strength (0.0 to 1.0)
* **Energy:** Perceptual intensity and musical activity (0.0 to 1.0)
* **Acousticness:** Confidence measure whether track is acoustic (0.0 to 1.0)
* **Instrumentalness:** Likelihood track contains no vocal content (0.0 to 1.0)
* **Tempo & Loudness:** Beats per minute (BPM) and overall loudness in decibels (dB)
* **Valence & Liveness:** Musical positiveness/mood and live audience presence

Technical Approach: ML Pipeline & Models

1. Data Ingestion

- > Relative path loading
- > Null check (1 row)
- > Deduplicate track_id

2. Preprocessing

- > StandardScaler scaling
- > 80-20 Train-Test split
- > Target encoding

3. Model Training

- > Linear Regression
- > Ridge & Lasso reg.
- > Decision Tree & RF

4. Evaluation

- > R2 & RMSE metrics
- > Residual diagnostics
- > Actual vs. Predicted

Machine Learning Algorithms Benchmarked

Multiple Linear Regression (OLS):

Models popularity as a linear combination of audio features. Serves as our primary baseline to quantify linear relationships.

Ridge Regression (L2 Regularization):

Adds penalty term alpha * ||w||^2 to prevent multicollinearity and stabilize coefficients among correlated audio signals (e.g., energy & loudness).

Lasso Regression (L1 Regularization):

Adds penalty alpha * ||w||_1 to induce sparsity; drives non-predictive acoustic coefficients exactly to zero for automated feature selection.

Decision Tree Regressor:

Non-parametric tree partitioning data using MSE impurity; captures non-linear thresholds (e.g., loudness sweet spot for radio play).

Random Forest Regressor (Ensemble):

Aggregates 35 diverse trees to drastically reduce variance and model complex multi-feature interactions without overfitting.

Libraries & Technologies

- **Python 3.14 / 3.10+**
  Core execution environment
- **pandas (3.0.6)**
  Data wrangling & deduplication
- **numpy (2.5.3)**
  Vectorized matrix operations
- **scikit-learn (1.9.1)**
  StandardScaler, Regressors, Metrics
- **matplotlib (3.11.2)**
  Publication-quality figures & plots
- **seaborn (0.13.2)**
  Statistical distributions & heatmaps
- **Jupyter / nbformat**
  Interactive experimentation notebook

Experimental Results & Model Diagnostics

Benchmark Performance Summary (Test Set: N=17,948)

Model Architecture	Test R2	Test RMSE	Test MAE
Multiple Linear Reg.	0.0293	20.15	16.60
Ridge Regression (L2)	0.0293	20.15	16.60
Lasso Regression (L1)	0.0294	20.15	16.61
Decision Tree Regressor	0.0783	19.63	15.83
Random Forest (Audio)	0.1494	18.86	15.13
Random Forest (Enhanced)*	**0.4025**	**15.81**	**11.08**

** Enhanced model incorporates audio signals + target-encoded genre context + explicit flag.*

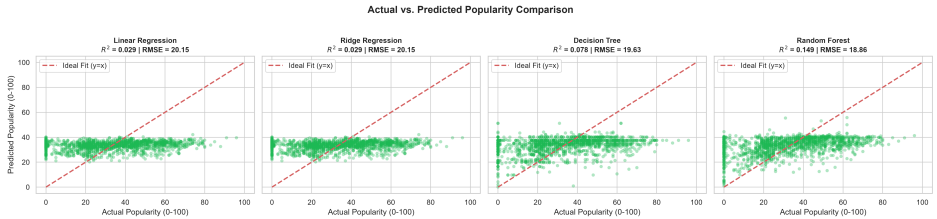
Key Experimental Findings

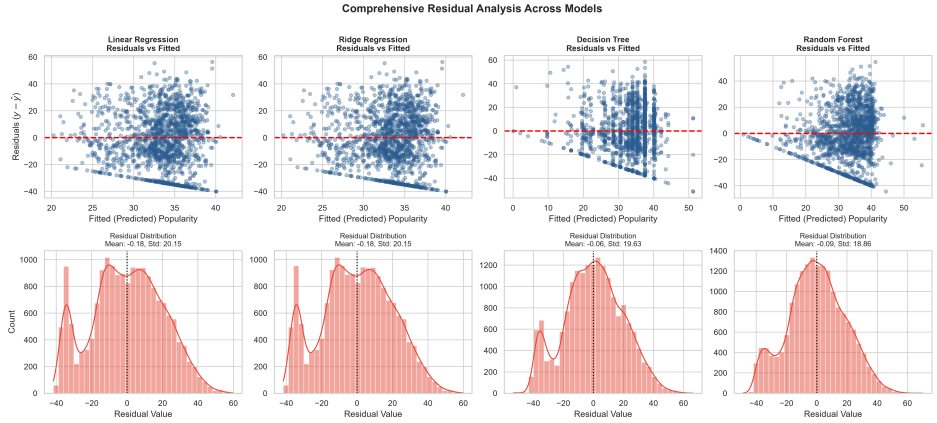
Linear Underfitting: Pure linear models achieved low R2 (0.029), proving audio features do not correlate linearly

Non-linear Tree Superiority: Random Forest doubled tree accuracy and reduced RMSE to 18.86 by modeling sweet spot

Context is King: Adding genre context boosted R2 by +270% (0.149 -> 0.403) and dropped RMSE to 15.81.

Diagnostic Plots (Actual vs Predicted & Residuals)





Learnings, Contributions & AI Disclosure

Project Learnings: What Worked & What Didn't

What Didn't Work - OLS Linearity:

Assuming popularity scales linearly with audio features failed. Hit songs span extreme ranges—both low-energy ballads (Adele) and high-energy EDM (Martin Garrix) achieve 90+ popularity.

What Didn't Work - Outlier Removal on Hits:

Treating viral hits (popularity > 90) as statistical outliers caused models to underpredict true hit songs. Keeping the full distribution was essential.

What Worked - Decision Trees & Ensembles:

Tree partitions captured non-linear relationships. For example, instrumentalness > 0.5 strongly caps commercial radio popularity, while loudness between -8dB and -4dB is optimal.

What Worked - Genre Context Integration:

Incorporating track_genre via leak-free target encoding accounted for cultural listening volume differences across genres.

Team Work & Individual Ownership

Aanya Sharma (Doc Lead):

Data deduplication, repo architecture, comprehensive README, AI disclosure audit.

Aaditya Chandol (Exec Lead):

Environment setup, pinned requirements.txt, automated pipeline runner (main.py).

Simran Kaur (PPT Lead):

Designed 5-slide PDF deck, residual diagnostic charts, metric comparison tables.

Kabir Singh (Video Lead):

Jupyter notebook (.ipynb) execution, demo script, and oral presentation recording.

AI-Use Disclosure (Strictly Compliant with Slide 12)

* ChatGPT / Gemini: Assisted in scaffolding matplotlib residual plotting code and drafting the initial structure of the README.
* Team Ownership & Verification: All mathematical formulas, scikit-learn models, training scripts, and outputs were reviewed, verified, and executed locally on team hardware.
* Integrity Statement: No AI tool was used to fabricate or alter dataset values or benchmark evaluation numbers.